



INFINITY WORKSPACE

Complete Product and Engineering Blueprint

A build-ready specification for an internal company operating system covering email, meetings, chat, tasks, files, people, approvals, administration, identity, security, infrastructure, and delivery.

DOCUMENT PURPOSE

Give the software engineering team a complete shared understanding of what to build, how the parts work together, which production controls are mandatory, and how to deliver the system safely in phases.

Prepared for engineering handoff | Version 1.0 | 26 August 2026

How to use this blueprint

This document defines the intended production system. The current browser prototype is a user-experience reference only; it is not a secure backend and must not be deployed as the production identity, mail, or storage service.

Audience

- Solution architect and technical lead.
- Backend, frontend, mobile, DevOps/SRE, QA, security, and data engineers.
- Business owner, operations lead, and implementation partner responsible for acceptance.

Normative language

- **MUST** means required for launch or security.
- **SHOULD** means strongly recommended; deviations require documented approval.
- **MAY** means optional or phase-dependent.
- Production Definition of Done requires code, tests, security controls, monitoring, backup, restore proof, and operational documentation.

System in one sentence

Infinity Workspace is a private, company-controlled digital workplace where authenticated employees communicate, meet, coordinate work, manage documents and approvals, and operate according to centrally administered permissions and audit policies.

Key decisions to confirm before implementation

Decision	Recommended default	Owner
Company domain	Use the verified corporate domain; infinity.lk is a prototype placeholder.	Business + IT
Deployment	Managed cloud in a primary region with encrypted backups in a separate account/region.	Technical lead
Mail strategy	Integrate a proven mail provider first; do not build SMTP reputation infrastructure in v1.	Business + architect
Meetings	Integrate a WebRTC provider or self-hosted LiveKit/Jitsi after capacity review.	Architect
Data residency	Select region based on legal, customer, and contractual obligations.	Legal + IT
User population	Confirm staff count, growth, contractors, guests, and mailbox size.	HR + IT

Vision, outcomes, and boundaries

The goal is not to clone every Microsoft 365 feature. The goal is to deliver the company workflows that matter, in one coherent and secure system, using proven infrastructure where building from scratch creates unnecessary operational risk.

Business outcomes

- Every employee receives a controlled company identity and mailbox under the verified company domain.
- Employees can sign in once and access only the modules, teams, records, and actions assigned to them.
- Internal communication, files, tasks, approvals, meetings, and decisions remain searchable and auditable.
- Administrators can onboard, suspend, recover, and audit accounts without engineering assistance.
- Operations continue during staff changes, device loss, mailbox migration, and service incidents.
- Company data is encrypted, backed up, retained, recoverable, and exportable according to policy.

In scope

Capability	Production intent
Identity and access	Company accounts, login, MFA, sessions, roles, module permissions, team/resource authorization, lifecycle management.
Mail	Send/receive, folders, threads, drafts, attachments, search, signatures, aliases, retention, quarantine and anti-abuse integration.
Calendar and meetings	Personal/team calendars, invitations, rooms, availability, reminders, video rooms, agenda, notes and recordings policy.
Chat and calls	Direct messages, channels, presence, files, reactions, mentions, unread state, search and optional voice/video.
Tasks and projects	Boards, assignments, dates, status, dependencies, comments, attachments, reminders and reporting.
Files and documents	Folders, versioning, permissions, previews, sharing, upload/download, recycle bin, retention and malware scanning.
People and operations	Directory, departments, announcements, approvals, audit reports and company-wide administration.

Explicitly not recommended for v1

- Building a new office-document editor comparable to Word or Excel.
- Operating a public internet mail-transfer network and spam reputation stack from scratch.
- Custom cryptography, custom password hashing, or custom WebRTC media infrastructure.
- Anonymous public file sharing or external guest access before governance is approved.
- AI features before data permissions, retention, audit, and human-review boundaries are mature.

Users, roles, and authorization

Authorization must be server-enforced. Hiding a menu in the frontend is usability only and is never a security control.

User types

Type	Purpose	Typical access
Super administrator	Emergency platform ownership; very small named group.	All settings, domain, retention, recovery and privileged audit.
Administrator	Daily user, group, policy and workspace administration.	Admin module plus assigned operational areas.
Manager	Leads a department/project and approves work.	Assigned teams, files, channels, reports and approval routes.
Staff	Normal employee account.	Personal mailbox plus assigned modules, teams and resources.
Auditor	Read-only compliance and audit access.	Selected audit logs, exports and reports; no content changes.
Guest/contractor	Optional future external collaboration.	Time-limited, sponsor-owned, explicitly shared resources only.
Service account	Machine integration, not interactive employee login.	Narrow API scopes, rotated credentials, no portal session.

Permission model

- RBAC: roles grant baseline capabilities such as user.create, mail.send, meeting.manage, file.share, approval.authorize, audit.export.
- Resource authorization: membership and ownership control access to specific mailboxes, channels, projects, folders, meetings and approval records.
- Policy conditions: department, data classification, device trust, network/location, MFA state, time window and employment status may further restrict actions.
- Deny by default: every API endpoint checks authenticated identity, tenant/company, capability and target resource before reading or changing data.
- Separation of duties: no user should create and finally approve the same high-value transaction; privileged-role changes require a second administrator or recorded approval.

Account lifecycle

Stage	Required behavior
Invite/create	Admin enters legal/display name, unique company email, employment metadata, manager, role and modules. System creates inactive or invited account and records audit event.
Activation	User follows a single-use, expiring invitation; establishes password/passkey and MFA; accepts policy; recovery codes are issued.
Active use	SSO session uses short-lived access token and rotating refresh/session record. Risk events can trigger step-up MFA.
Change	Role, manager, department and group changes are effective immediately and audited with before/after values.
Suspend	Revoke sessions and API tokens immediately; preserve mailbox/files under retention; block sign-in and outbound mail.
Offboard	Transfer owned resources, configure mailbox delegation/forwarding if authorized, preserve records, then delete/anonymize only after retention permits.

User journeys and acceptance flows

Administrator creates an employee

1. Open Admin > Portal Users.
2. Enter name and verified-domain address; system checks uniqueness and allowed domain.
3. Select access level, department, manager, groups and specific modules.
4. Send a single-use invitation; never disclose a permanent password in plain email.
5. User completes activation and MFA; admin sees Active state and immutable audit record.

Employee starts the day

1. Sign in with company identity and MFA when required.
2. Command dashboard shows unread mail/chat, today meetings, assigned tasks, pending approvals and alerts.
3. Search returns only content the user is authorized to view.
4. Actions update in real time and remain available after refresh/device change.

Manager approves a request

1. Request is created from mail, task, procurement or HR workflow.
2. Routing engine identifies approver by amount, department and policy.
3. Approver sees context and attachments, then approves/rejects/returns with comment.
4. Decision is immutable, timestamped, notified, and reflected in audit/reporting.

Administrator suspends a user

1. Admin confirms suspension and reason.
2. All sessions, refresh tokens, API tokens and active meeting/chat connections are revoked.
3. Mailbox, files, tasks and ownership are preserved and transferred per policy.
4. Sign-in returns a neutral suspended-account response and support path.

Cross-module event examples

Event	Downstream behavior
Meeting scheduled	Calendar event, participant notifications, room reservation, reminders, chat link and optional task/agenda creation.
File attached to chat	Upload service scans and versions object; chat stores a reference; authorization follows channel plus file policy.
Task assigned	Task event creates notification, updates dashboard counter and can schedule reminder jobs.
User suspended	Identity event revokes sessions, closes real-time sockets, disables outbound mail and triggers ownership review.

Functional requirements by module

All modules share identity, notifications, search, files, audit, retention, accessibility, responsive behavior, and a common design system.

Command dashboard

Personalized overview of unread items, meetings, tasks, approvals, announcements, storage/risk alerts, and quick actions. Widgets must respect permissions and load independently so one failed service does not blank the page.

Mail

Mailbox/folder model, conversation threads, compose/reply/forward, To/CC/BCC, drafts, signatures, attachments, labels, flags, archive, trash, spam/quarantine, delivery status, aliases/shared mailboxes, full-text search, rules and retention. MIME parsing and sending must use proven libraries/providers.

Calendar and meetings

Day/week/month views, recurring events, time zones, attendees, RSVP states, free/busy, room/resources, reminders, agendas, meeting links, waiting room, host controls, screen sharing, captions/recording policy, notes and attendance. Store all timestamps in UTC plus IANA time-zone identifiers.

Chat

1:1 and group direct messages, channels, membership roles, replies/threads, mentions, reactions, edits/deletes under policy, typing/presence, read markers, pinned items, attachments, search, retention and moderation/audit. Real-time delivery uses WebSocket; database remains source of truth.

Tasks and projects

Boards/lists, task title/description, assignee, watchers, priority, status, due date, recurrence, dependencies, checklists, comments, files, labels, activity history, reminders, saved views and project-level permissions.

Files

Folder hierarchy, upload/download, multipart/resumable upload, checksum, object metadata, version history, preview generation, antivirus scanning, classification, access grants, recycle bin, legal hold, quotas and audited sharing. Binary objects belong in object storage, not database blobs.

Module requirements (continued)

People

Searchable directory, profiles, title, department, manager, contact details, status, skills optional, org structure and groups. Sensitive HR data belongs in a separate HR system or tightly restricted schema, not the general directory.

Approvals

Configurable request types, forms, conditional routing, amount thresholds, sequential/parallel approvers, delegation, expiry/escalation, immutable decision history, attachments, notifications, reports and cancellation rules.

Announcements

Targeted company/department/group broadcasts, scheduling, acknowledgements, attachments, expiration, priority banners, delivery/read statistics and administrator moderation.

Administration

Users, groups, roles, module policies, domains, branding, mail settings, rooms/resources, storage quotas, retention, audit, integrations, security status, backup/restore visibility and license/cost reporting.

Recommended production architecture

Start as a modular monolith with clearly separated domains and asynchronous workers. This reduces delivery and operational complexity while preserving boundaries that can become services when scale or ownership justifies it.

CLIENTS

Responsive web application (React/TypeScript) | optional mobile/PWA | administrator console

EDGE AND ACCESS

DNS + CDN + WAF + TLS | reverse proxy/API gateway | rate limiting | request IDs

APPLICATION

Identity/Auth | Mail | Calendar/Meeting | Chat | Tasks | Files | People | Approvals | Admin | Search API

ASYNC AND REAL-TIME

WebSocket gateway | job workers | event/outbox dispatcher | notifications | mail ingest/send | preview/virus scan | search indexing

DATA AND INTEGRATIONS

PostgreSQL | Redis | object storage | search engine | SMTP/IMAP or mail API | WebRTC provider | SMS/push | observability

Technology baseline

Layer	Recommended options	Notes
Web client	React + TypeScript + Vite/Next.js; accessible component system	Keep API client typed; route and component gates use server-provided capabilities.
Backend	TypeScript/NestJS, Kotlin/Spring, .NET, or Go	Choose team expertise. Enforce domain modules, dependency boundaries and one migration system.
Primary data	PostgreSQL	Transactions, relational authorization, audit references, JSONB only where justified.
Cache/jobs	Redis + BullMQ/Sidekiq-equivalent	Cache, rate limits, ephemeral presence and background jobs; never source of truth for records.
Files	S3-compatible object storage	Private buckets, short-lived signed URLs, server-side encryption, versioning and lifecycle policies.
Search	OpenSearch/Elasticsearch or PostgreSQL FTS for smaller v1	Every indexed document carries tenant/resource ACL metadata.
Real-time	WebSocket gateway + pub/sub	Authenticate connection; authorize subscriptions; persist messages before acknowledgement.
Deployment	Containers on managed Kubernetes/ECS/App Service or managed PaaS	Prefer managed services until scale/requirements justify self-hosting.

Domain boundaries and service responsibilities

Domain	Owns	Must not own
Identity	Users, credentials references, MFA, sessions, invitations, recovery, login policy.	Mailbox content, project permissions or business approvals.
Authorization	Roles, capabilities, group membership, policy evaluation and resource grants.	Frontend-only permission decisions.
Mail	Mailboxes, messages, MIME metadata, folders, drafts, delivery state and provider sync.	Binary attachment storage or employee master data.
Calendar	Events, recurrences, attendees, free/busy, resources and reminders.	WebRTC media packets.
Chat	Rooms, membership, messages, threads, reactions, read state and moderation.	Durable binary files.
Tasks	Projects, tasks, assignments, dependencies, activity and saved views.	Approval policy configuration.
Files	Object metadata, versions, checksums, grants, scanning and retention.	Raw identities or credentials.
Approvals	Definitions, instances, steps, decisions, delegation and escalation.	Editing source financial/HR transactions.
Audit	Append-only security and business event trail plus exports.	Mutable operational truth.

Communication rules

- Synchronous HTTP/JSON or gRPC is acceptable for user-facing commands that require an immediate response.
- Background work uses durable jobs/events. Apply the transactional outbox pattern so database commits and emitted events cannot diverge.
- Events are versioned, idempotent, timestamped, traceable, and include company/tenant ID plus actor where applicable.
- No domain reads another domain database table as an informal API. In a modular monolith, enforce this through package/repository boundaries and integration tests.
- External providers are behind adapters so mail, meeting, storage and notification vendors can be replaced.

Example command lifecycle

Step	Create employee account
1	Admin client sends POST /v1/users with profile, access level, modules and idempotency key.
2	API authenticates admin, verifies user.create capability and validates verified company domain.
3	Identity transaction creates invited user, access assignments, audit record and outbox event.
4	Worker consumes user.invited, creates provider mailbox if configured and sends single-use activation link.
5	User establishes credentials/MFA; activation rotates invitation token and emits user.activated.
6	Directory, search and notification projections update asynchronously; admin sees completion state.

Data model

Every business table must include a stable UUID, company/tenant ID, timestamps, and version/concurrency fields where simultaneous edits are possible. Use soft deletion only where recovery or retention requires it; otherwise model explicit lifecycle states.

Entity group	Important fields / rules
companies	id, name, verified_domains, region, status, settings
users	id, company_id, email, display_name, status, department_id, manager_id, locale, timezone
identities	user_id, password_hash/provider_subject, MFA methods, last_auth_at; isolate and encrypt sensitive fields
roles / permissions / grants	role, capability, subject, resource_type, resource_id, effect, conditions
sessions / tokens	hashed token identifier, device, IP/risk metadata, expiry, revoked_at
mailboxes / messages	mailbox, provider IDs, thread, headers, body references, folder, flags, delivery and retention state
calendar_events / attendees	organizer, recurrence rule, UTC times, timezone, attendee status, room, meeting provider ID
chat_rooms / members / messages	room type, member role, message sequence, body, parent, edited/deleted state, read cursor
projects / tasks / comments	project, assignee, status, priority, dates, dependency, labels, activity
files / versions / grants	object key, checksum, MIME, size, scanner state, classification, owner, retention, access grants
approval_definitions / instances / steps	schema version, request data, route, approver, decision, comments, due/escalation
notifications	recipient, type, channel, payload reference, read/delivery status, dedupe key
audit_events	actor, action, resource, result, IP/device, correlation ID, before/after redaction-safe metadata

Data integrity rules

- Unique lower-cased email per company and verified domain; preserve original display casing separately if needed.
- Foreign keys and constrained state enums; prohibit orphaned memberships, attachments, approval steps and ownership.
- Optimistic concurrency for messages/tasks/events/settings edited by multiple clients.
- Audit content must avoid credentials, access tokens, full message bodies and unnecessary personal data.
- Use database transactions for state changes and corresponding audit/outbox records.
- Partition or archive high-volume audit, message and notification tables based on measured growth, not premature assumptions.

API and real-time contract

API conventions

- Versioned HTTPS endpoints under /api/v1; OpenAPI is the contract and generates typed clients.
- JSON request/response, RFC 3339 timestamps, UUID identifiers, cursor pagination, consistent filtering/sorting.
- Standard error envelope: code, human-safe message, field errors, correlationId; never expose stack traces.
- Idempotency-Key required for invitation, send mail, create meeting, approval decision and other retry-sensitive commands.
- ETag/version preconditions for conflicting updates. 401 means unauthenticated; 403 means authenticated but forbidden.
- Rate limits by company, user, IP, endpoint and provider constraints; include retry information.

Endpoint group	Contract intent
POST /auth/login, /auth/mfa/verify	Create authenticated session; rate limit and risk-check.
POST /users, PATCH /users/{id}	Create/invite and update lifecycle/access; administrator only.
POST /users/{id}/suspend	Immediate session/token revocation and suspension workflow.
GET /me, /me/capabilities	Current profile, tenant, groups and effective capabilities.
GET/POST /mail/messages	List authorized mailbox content and send mail using provider adapter.
GET/POST /calendar/events	Calendar query and event creation with recurrence/time-zone validation.
GET/POST /chat/rooms/{id}/messages	Paginated history and durable message creation.
GET/POST/PATCH /projects/{id}/tasks	Task query, create and controlled update.
POST /files/uploads	Create multipart upload; finalization verifies size/checksum and starts scan.
POST /approvals/{id}/decisions	Approve/reject/return with idempotency and separation-of-duties check.
GET /search?q=	Permission-filtered cross-module results and typed facets.
GET /audit/events	Privileged, filtered, paginated, redacted audit access.

WebSocket events

Channel	Examples	Rules
user:{id}	notification.created, session.revoked	Private to authenticated user; reconnect with cursor.
room:{id}	message.created, message.updated, typing, presence	Authorize room membership before subscribe; typing/presence are ephemeral.
meeting:{id}	participant.joined, agenda.updated	Authorize attendee/host role; media handled by provider.
project:{id}	task.created, task.updated	Authorize project membership and field-level visibility.

Email architecture

Email is the highest-risk operational subsystem. The recommended launch path is a managed provider with API/SMTP integration, while Infinity Workspace owns the user experience, authorization, synchronization metadata, retention policy and audit trail.

Provider options

Approach	Advantages	Risks / use
Managed mailbox provider	Fastest reliable delivery, spam/abuse, DKIM, inbound routing, reputation and support.	Recommended for v1. Confirm mailbox, retention, region, API and cost requirements.
Transactional mail provider only	Good for application notifications and outbound transactional mail.	Not a complete employee mailbox. Requires separate inbound/mailbox store.
Self-hosted mail	Maximum control.	High operational burden: SMTP reputation, queues, spam, malware, abuse, DNS, patching, deliverability and 24/7 monitoring. Avoid unless expert team and clear requirement.

DNS and domain requirements

- Verify domain ownership.
- Configure MX for inbound routing, SPF for authorized senders, DKIM signing with rotation, and DMARC reporting/enforcement.
- Configure bounce/complaint handling, abuse contacts, reverse DNS where applicable, and TLS reporting/MTA-STS if supported.
- Never claim that creating a portal user automatically creates a working internet mailbox until the provider provisioning call succeeds.

Mail processing flow

- Outbound: authorize sender -> persist draft/outbox state -> MIME build -> malware/DLP checks -> provider submit -> webhook delivery/bounce updates -> notify/audit.
- Inbound: provider receives/scans -> signed webhook or IMAP sync -> deduplicate provider message ID -> parse MIME safely -> store metadata/body and private attachment references -> index with ACL -> notify.
- HTML email is sanitized; remote images are blocked or proxied based on privacy policy; dangerous attachments are quarantined.
- Provider webhook signatures, replay windows and idempotency are mandatory. Mail retries use backoff and a dead-letter queue.

Meetings, calls, and presence

Meeting strategy

Use WebRTC through a proven provider or a well-operated platform such as LiveKit/Jitsi. Infinity Workspace creates meetings and authorization tokens; the media platform handles routing, NAT traversal, adaptive bitrate, screen share, device negotiation and recording infrastructure.

Component	Responsibility
Calendar service	Event, attendee, recurrence, room/resource, reminders, permissions and meeting-link lifecycle.
Meeting token service	Short-lived signed tokens containing room, user, company and host/participant permissions.
Media provider/SFU	Audio/video transport, TURN/STUN, screen share, participant quality and optional recording.
Recording worker/storage	Policy check, consent indicator, encrypted recording, transcript, retention, access grants and audit.
Presence service	Ephemeral online/away/in-meeting state with privacy controls; Redis TTL and socket heartbeat.

Mandatory behaviors

- Waiting room and host admission options.
- Mute/camera/screen-share controls and device selection.
- Participant authorization is rechecked when a user is suspended or removed.
- Recording requires visible notice/consent appropriate to law and company policy.
- Meeting metadata remains usable if media provider is unavailable; degraded state is clear.
- Capacity test concurrent meetings, participants, TURN bandwidth and recording load before launch.

Files, search, and content safety

Upload pipeline

- Client requests upload session after file.create authorization and quota check.
- API issues short-lived multipart signed URLs scoped to exact key, type and size.
- Client uploads directly to private object storage and finalizes with checksum.
- Worker verifies object, MIME-sniffs content, scans malware, generates preview/thumbnail and extracts safe searchable text.
- File remains Processing/Quarantined until checks pass; users never receive a public permanent object URL.
- Version is created atomically and file.created/file.versioned event updates search and notifications.

Search authorization

- Index mail, chat, files, people, tasks, meetings and announcements as separate document types.
- Every index record includes company ID, resource ID, classification and ACL/group tokens.
- Query layer computes current access filters server-side; never return unauthorized hits and hide them in the client.
- Permission changes and suspensions trigger urgent index updates; sensitive resources may require source-of-truth recheck before result return.
- Provide result snippets with HTML escaping and no untrusted active content.

Retention and deletion

State	Expected behavior
Active	Normal use; versioning and audit apply.
Recycle bin	User-hidden but recoverable for configured period; ownership and ACL retained.
Retention hold	Deletion blocked or logically hidden while compliance copy remains immutable.
Expired	Asynchronous purge from primary/object/search/cache plus tombstone event; backup expiry follows documented schedule.

Security and privacy requirements

Security is part of the product definition. These controls are launch requirements, not a future hardening phase.

Control area	Minimum implementation
Authentication	Argon2id/bcrypt password hashing through vetted library; MFA/passkeys; breached-password check; secure recovery; no plaintext passwords.
Sessions	HttpOnly Secure SameSite cookies or secure token pattern; CSRF protection; rotation; device/session list; immediate revocation.
Authorization	Server checks on every endpoint and real-time subscription; deny by default; tenant/company scoping in queries.
Transport/data	TLS 1.2+; managed certificates; encryption at rest; KMS-managed secrets/keys; field encryption for highly sensitive values.
Application	Input/schema validation, output encoding, CSP, safe email HTML, SSRF protection, file scan, dependency and secret scanning.
Abuse protection	Rate limits, lockout/risk controls, spam limits, upload quotas, webhook verification, bot/credential-stuffing monitoring.
Administration	MFA mandatory, step-up for destructive/high-risk operations, optional dual control, IP/device policy and privileged audit.
Audit	Append-only, tamper-evident or immutable storage export, synchronized clocks, correlation IDs, redaction and access monitoring.
Secrets	Cloud secrets manager; no secrets in code, browser bundle, logs, tickets or CI output; rotation and access review.
Privacy	Data inventory, purpose limitation, minimization, retention schedule, access/export/correction workflows and lawful recording policy.

Threats to explicitly test

- Cross-company/tenant ID enumeration and broken object-level authorization.
- Role escalation, stale permission caches and hidden frontend routes called directly.
- Session fixation/theft, invitation token reuse, recovery abuse and MFA bypass.
- Stored XSS in chat/mail/file names, malicious HTML email, unsafe previews and content-type confusion.
- Malware uploads, zip bombs, oversized payloads, signed-URL abuse and public-bucket exposure.
- Webhook forgery/replay, provider credential leakage, queue poisoning and idempotency failures.
- Search results, exports, notifications or audit logs leaking restricted content.

Infrastructure, environments, and deployment

Environment separation

- Local development uses synthetic data and provider sandboxes.
- Development, staging and production use separate cloud accounts/projects, networks, databases, buckets, keys, domains and credentials.
- Production personal/company data must never be copied to non-production without approved masking.
- Infrastructure is version-controlled as code; manual production changes are exceptional and audited.

Reference deployment

Tier	Components
Edge	Authoritative DNS, CDN/static assets, WAF, DDoS protection, TLS termination and rate limiting.
Application	At least two stateless API instances across zones; WebSocket instances; worker pools by workload.
Data	Managed PostgreSQL with multi-zone HA and PITR; managed Redis; private object storage; managed search.
Network	Private subnets for data, restricted security groups/firewalls, controlled egress, no public database endpoints.
Operations	Central logs, metrics, traces, alerting, on-call notifications, status page/runbooks and cost monitoring.

CI/CD pipeline

- Pull request: format/lint, type check, unit tests, API contract tests, migration validation, dependency/license/secret scans and build.
- Ephemeral or shared test deployment: integration, accessibility, browser, authorization matrix and provider sandbox tests.
- Staging: production-like smoke, load and recovery checks; signed image artifacts and generated SBOM.
- Production: approval gate, rolling/canary deployment, automated health checks, migration compatibility, rollback plan and release annotation.
- Database migrations follow expand/migrate/contract so old and new application versions can overlap safely.

Reliability, backup, and incident response

Suggested initial service objectives

Measure	Initial target	Measurement
Portal/API availability	99.9% monthly excluding approved maintenance	Synthetic checks + request success SLI.
Interactive API latency	95% under 500 ms; 99% under 1.5 s	Server request traces excluding provider-dependent jobs.
Chat delivery	95% under 2 s when connected	Persist-to-client acknowledgement metric.
Notification job	95% under 60 s	Event timestamp to delivery attempt.
Recovery point objective	15 minutes for transactional data	PITR/WAL and backup configuration proof.
Recovery time objective	4 hours for core operations	Quarterly restore exercise.

Backup requirements

- Automated encrypted PostgreSQL backups plus point-in-time recovery; retention aligned to policy.
- Object storage versioning/replication and deletion protection appropriate to threat model.
- Configuration, infrastructure code and encryption-key recovery procedures are included; a database dump alone is not a system backup.
- Backups are isolated from normal administrator credentials and protected from ransomware-style deletion.
- Restore tests run on schedule and record actual recovery time, data validation and remediation.

Incident response

- Severity matrix, on-call ownership and escalation contacts.
- Runbooks for login outage, mail queue failure, provider webhook backlog, database saturation, object-storage exposure, account compromise and data loss.
- Ability to revoke sessions/tokens, block providers, suspend outbound mail, disable public integrations and place audit retention hold.
- Post-incident review records timeline, impact, root cause, corrective actions, owners and deadlines without blame.

Observability and audit

Telemetry

Signal	Examples	Rules
Metrics	Request rate/error/latency, active sockets, queue depth/age, mail delivery, provider errors, DB pool, cache, storage, scan time.	Aggregate labels; avoid high-cardinality user/message IDs.
Logs	Structured JSON with timestamp, service, environment, severity, request/correlation ID, actor ID where appropriate and result.	No passwords, tokens, message bodies, attachment content or unnecessary PII.
Traces	Edge -> API -> database/queue -> worker -> provider.	Sample intelligently; preserve errors and slow traces; redact sensitive attributes.
Audit	Login, MFA, user lifecycle, role/grant, export, share, retention, approval and admin setting changes.	Business/security record with controlled access and longer retention.

Alerting principles

- Alert on user/business impact and exhausted error budgets, not every isolated error.
- Queue age, mail bounce spikes, authentication failures, privilege changes, backup failure, scan failure and storage exposure require targeted alerts.
- Every alert links to dashboard and runbook and has a named owning team.
- Test alert routing and escalation; stale alerts and false-positive noise are reliability defects.

Frontend and experience requirements

Application shell

- Persistent module navigation, current module title, global search, notifications, call shortcut and account/session controls.
- Desktop, laptop, tablet and phone layouts with no document-level horizontal overflow. Wide data tables scroll inside their region.
- Deep links and browser history for modules/resources; refresh restores view without losing authorized state.
- Loading, empty, offline, error, forbidden, suspended and degraded-provider states are designed for every module.
- Optimistic updates only where rollback is clear; durable records reconcile with server responses and real-time events.

Accessibility baseline

- Target WCAG 2.2 AA.
- Keyboard operation and visible focus for all workflows; logical heading/landmark structure and skip navigation.
- Accessible names for icon controls; status announcements; labels and field-level errors; no color-only meaning.
- Color contrast, 200% zoom, reduced motion, touch target size and screen-reader tests.
- Email/chat editors and complex boards need tested keyboard alternatives, not drag-only interaction.

Client architecture

- Typed API layer generated from OpenAPI where practical.
- Server state via a query cache with invalidation/realtime reconciliation; local UI state kept separate.
- Feature modules align with backend domains; shared design system contains tokens, controls, forms, feedback and responsive primitives.
- Route guards improve navigation but APIs remain authoritative. Current user/capabilities are fetched at startup and refreshed after changes.
- Never store permanent credentials or sensitive message/file data in localStorage. Use secure sessions and bounded offline caches only if approved.

Testing and quality strategy

Level	Required coverage
Unit	Domain rules, validators, policy evaluation, recurrence, routing, parsing helpers and state transitions.
Integration	Database repositories/transactions, outbox, queues, object storage, search ACL, provider adapters and webhook verification.
API contract	OpenAPI conformance, standard errors, pagination, idempotency, concurrency and backwards compatibility.
Authorization	Matrix tests for every capability/resource role including cross-company IDs, suspension and stale access.
End-to-end	Invite/activate/login/MFA; send/receive mail; meeting; chat; task; file scan/version; approval; suspend/revoke.
Security	SAST, dependency/secret/container/IaC scans, DAST, penetration test before launch and remediation verification.
Performance	Expected and peak users, mail ingest spikes, concurrent sockets/meetings, large uploads, search, queue recovery and soak.
Resilience	Provider timeout, duplicate event, worker crash, cache loss, database failover, full queue, partial deployment and restore.
UX/accessibility	Responsive screenshots at defined widths, keyboard/screen reader, browser support, empty/error/slow states.

Release gates

- No unresolved critical/high security issue unless formally risk-accepted by accountable owner.
- No failing authorization, migration, backup/restore or core end-to-end test.
- Monitoring, alerting, runbooks and rollback verified in staging.
- Data migration reconciliation and business acceptance signed.
- Known limitations documented with owner, risk, workaround and target date.

Delivery roadmap

Exact dates depend on team size, provider choices and integrations. The sequence below reduces risk by establishing identity, data controls and operating foundations before expanding collaboration features.

Phase	Scope	Exit criteria
0 - Discovery and decisions	Requirements workshops, domain/provider/data residency, users/volume, policies, UX prototype, architecture threat model and backlog.	Signed requirements, decisions, cost model, delivery plan and acceptance owners.
1 - Platform foundation	Cloud environments, CI/CD, identity, MFA, sessions, RBAC/resource grants, audit, design system, shell, observability and backup.	Admin can securely create/activate/suspend user; authorization and restore tests pass.
2 - Core operations	Directory, tasks/projects, files, notifications, announcements, approvals and global search.	Complete internal workflows with audit, permissions, scanning and responsive UX.
3 - Mail and calendar	Domain/provider provisioning, mail sync/send, folders/search, calendar, rooms, invitations/reminders.	Pilot users exchange real mail and schedule meetings with delivery/security monitoring.
4 - Chat and meetings	Channels/DM, presence, WebSockets, WebRTC provider, agenda/notes and optional recording.	Concurrency, reconnect, moderation, retention and provider-degradation tests pass.
5 - Migration and pilot	User/mail/file import, coexistence, training, support, telemetry and controlled departments.	Reconciliation signed; pilot SLO/security/UX targets met.
6 - Company rollout	Waves, support/on-call, decommission plan, governance cadence and cost optimization.	All departments live, recovery validated, ownership transferred to operations.

Team shape

- Product owner/business analyst with authority to resolve workflow decisions.
- Technical lead/architect, 2-4 backend engineers, 1-2 frontend engineers, DevOps/SRE, QA automation and security support.
- UX/product designer for information architecture, responsive behavior, accessibility and user testing.
- Specialist support for mail deliverability/DNS and WebRTC if those systems are not fully managed.

Migration and launch

Migration inventory

- Users, aliases, groups, shared mailboxes, distribution lists and domains.
- Mail folders/messages/attachments, calendar events, contacts, files/folders/permissions and task/project records.
- Retention/legal holds, delegated access, resource mailboxes and automation/integration dependencies.
- Unsupported or corrupt items are reported; source counts, imported counts, skipped counts and checksums reconcile.

Cutover approach

- Run discovery and dry-run migrations first; do not use the final cutover as the first full import.
- Pilot a representative department and retain rollback/coexistence path.
- Lower DNS TTL before mail cutover; schedule and communicate MX/DKIM/SPF/DMARC changes; monitor queues, bounces and inbound delivery.
- Use incremental delta migration after bulk import, then freeze/cut over within an approved window.
- Provide user guides, administrator runbooks, helpdesk scripts and escalation route.

Launch readiness checklist

Area	Must be true
Identity	All active users provisioned; administrators use MFA; recovery and suspension tested.
Mail/domain	Domain verified; DNS passes; inbound/outbound/bounce/complaint tests pass; rollback documented.
Data	Migration reconciled; permissions sampled; retention/legal approvals recorded.
Reliability	Backups successful; restore tested; dashboards/alerts/on-call live.
Security	Threat model and penetration remediation complete; secrets rotated; privileged access reviewed.
Support	Training delivered; helpdesk and vendor escalation ready; status communications prepared.

Production acceptance criteria

Capability	Acceptance result
Account creation	Admin can create a unique verified-domain user, assign modules/resources, send expiring activation and see provider mailbox status.
Login/access	Active user completes secure login/MFA; suspended user cannot authenticate; direct API calls to forbidden modules return 403.
Session revocation	Suspension closes sessions and real-time connections within target time and invalidates refresh/API tokens.
Mail	Authorized user sends/receives with attachments; SPF/DKIM/DMARC align; delivery/bounce reflected; HTML/attachments are safely processed.
Meetings	Users schedule across time zones, RSVP, join authorized room, control media and receive reminders; recurrence is correct.
Chat	Durable ordered messages, reconnect/catch-up, membership enforcement, unread/read state, retention and search work.
Tasks	Assignment, due dates, state changes, comments, notifications and activity history persist and enforce project access.
Files	Private resumable upload, checksum, malware scan, preview, versioning, grants, recycle/restore and audit pass.
Approvals	Configured routing, delegation/escalation, idempotent decision and immutable history meet policy.
Search	Authorized results across modules; restricted content is absent from result, snippet, count and suggestion.
Operations	SLO dashboards, alerts, runbooks, backup and measured restore exercise are complete.
Responsive/accessibility	Core workflows pass desktop/tablet/390px checks, keyboard access and WCAG 2.2 AA verification.

Risks, trade-offs, and open decisions

Risk / decision	Consequence	Mitigation / required answer
Building mail hosting from scratch	Deliverability, abuse and 24/7 operational risk.	Use managed provider for v1; reassess only with expert team and business case.
One application vs microservices	Premature services slow delivery and create distributed failure modes.	Modular monolith plus workers/events; extract only with measured scale/ownership need.
Local browser account storage	Prototype passwords can be read/modified and are not secure.	Production identity in backend/IdP with hashing, MFA, server authorization and secure sessions.
Recording meetings	Privacy, consent, storage and legal obligations.	Disabled by default until policy, notice, access and retention are approved.
External sharing/guests	Data leakage and lifecycle risk.	Launch internal-only; add sponsor, expiry, classification and audit before enabling.
Search ACL drift	Restricted content may leak in results.	ACL-aware index, urgent permission events, source recheck for sensitive results and tests.
Provider dependency	Mail/meeting outage or price/API change.	Adapter boundaries, export path, degraded mode, status monitoring and contractual review.
Scope comparable to M365	Unbounded cost and schedule.	Prioritize company workflows; integrate document editing/provider capabilities instead of cloning all features.

Questions the engineer must resolve with the owner

- What is the real verified company domain and who controls its DNS?
- How many users, shared mailboxes, guests, simultaneous meetings, total files and yearly growth?
- Which country/region must hold data and which laws/contracts govern retention and recordings?
- Which modules are day-one requirements and which may follow after core identity/operations?
- Is existing mail/data available for migration, and from which formats/providers?
- What approval limits, departments, separation-of-duties and audit retention rules apply?
- What availability/recovery commitments can the company fund and operate?
- Who owns production operations, security response, mail abuse handling and user support after handoff?

Engineer handoff checklist

The engineer should not start by converting the prototype localStorage logic into a hosted application. Begin with decisions, domain models, security boundaries and provider proof-of-concepts.

First 30 engineering actions

#	Action
1	Confirm business owner, technical owner and acceptance authority.
2	Workshop modules and rank must/should/later.
3	Confirm verified domain and DNS control.
4	Measure users, data and concurrency.
5	Select cloud region and data residency.
6	Decide managed mail provider and prototype mailbox provisioning/webhooks.
7	Decide meeting provider and prototype token/join flow.
8	Approve modular-monolith stack.
9	Create architecture decision records.
10	Threat-model identity, files, mail and cross-tenant access.
11	Define role/capability/resource permission matrix.
12	Define account invitation, MFA, recovery, suspension and offboarding.
13	Create OpenAPI conventions and error envelope.
14	Draft PostgreSQL schema and migration approach.
15	Implement audit and transactional outbox foundations.
16	Provision isolated dev/staging/prod infrastructure as code.
17	Configure CI security and quality gates.
18	Implement secure session/auth integration.
19	Implement server-side authorization middleware/policy engine.
20	Build user administration and activation flow.
21	Add observability and correlation IDs from the first endpoint.
22	Set up private object storage and scanning proof-of-concept.
23	Build design system and responsive app shell.
24	Implement notifications and durable job framework.
25	Implement first vertical slice: create user -> activate -> login -> authorized dashboard.
26	Automate authorization matrix and end-to-end tests.
27	Define backup, PITR and restore drill.
28	Plan migration discovery and reconciliation reports.
29	Pilot with synthetic data before real employee data.
30	Review security, cost and operational readiness before expanding scope.

Appendix: capability catalogue

Domain	Candidate capabilities
identity	user.read, user.create, user.update, user.suspend, user.reactivate, role.assign, session.revoke
mail	mail.read, mail.send, mail.delete, mailbox.delegate, shared_mailbox.manage, quarantine.manage
calendar	calendar.read, event.create, event.update, event.cancel, room.manage, freebusy.read
meeting	meeting.join, meeting.host, participant.manage, recording.start, recording.read, transcript.read
chat	room.create, room.join, member.manage, message.send, message.edit, message.delete, moderation.manage
tasks	project.create, project.manage, task.create, task.assign, task.update, task.delete, report.read
files	file.read, file.create, file.update, file.delete, file.share_internal, file.restore, legal_hold.manage
approvals	request.create, request.cancel, decision.make, delegation.manage, definition.manage, approval.report
admin/audit	settings.read, settings.update, integration.manage, retention.manage, audit.read, audit.export, backup.view

Capability implementation rule

Capabilities describe an action category. Final authorization also evaluates company/tenant, target resource, membership/ownership, classification, lifecycle state, policy conditions and separation-of-duties. A role name alone must never grant implicit access to every record.

Appendix: non-functional requirement checklist

Attribute	Definition of success
Security	MFA, least privilege, secure sessions, encryption, scanning, audit, testing and response controls defined in Section 12.
Availability	No single application instance; managed HA data; graceful provider degradation; SLOs and health checks.
Performance	Targets defined per workflow; pagination; async provider/file work; load tests with realistic data distribution.
Scalability	Stateless API, horizontally scalable workers/sockets, direct object upload, partition only when measurements justify.
Maintainability	Domain boundaries, typed contracts, migrations, automated tests, ADRs, code ownership and dependency update process.
Usability	Consistent navigation, clear language, quick common workflows, complete states, responsive design and accessibility.
Interoperability	Provider adapters, standard MIME/iCalendar where applicable, documented APIs, export formats and webhook versioning.
Recoverability	PITR, object versions, isolated backups, restore exercises, ownership and documented RPO/RTO.
Compliance/privacy	Data inventory/classification, retention, legal hold, recordings policy, access/export and audit controls.
Operability	Metrics/logs/traces, runbooks, alerts, feature flags, rollback, cost dashboards and named on-call ownership.

END OF BLUEPRINT

This specification should be placed under version control. Every material scope, architecture, provider, security, retention or acceptance change must update the relevant section and record an architecture/product decision.